

Java Stream API — Simple Interview Programs

20 beginner-friendly Stream API programs for SDET/Java interviews. Each exercise includes a problem statement, complete runnable Java code, and expected output.

1. Filter Strings by Length

Problem Statement: Given a list of product names, find only the names whose length is greater than 4.

Java Program:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample1 {
    public static void main(String[] args) {

        List<String> titles =
            Arrays.asList("Zara", "Nike", "Apple");

        List<String> result = titles.stream()
            .filter(s -> s.length() > 4)
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

Expected Output: [Apple]

2. Convert Strings to Uppercase

Problem Statement: Given a list of names, convert every name to uppercase using Stream API.

Java Program:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample2 {
    public static void main(String[] args) {

        List<String> names =
            Arrays.asList("swapnil", "amit", "rahul");

        List<String> result = names.stream()
            .map(String::toUpperCase)
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

Expected Output: [SWAPNIL, AMIT, RAHUL]

3. Filter and Map Together

Problem Statement: Given product titles, keep titles with more than 4 characters and convert them to uppercase.

Java Program:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample3 {
    public static void main(String[] args) {

        List<String> titles =
            Arrays.asList("Zara", "Nike", "Apple", "Samsung");

        List<String> result = titles.stream()
            .filter(s -> s.length() > 4)
            .map(String::toUpperCase)
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

Expected Output: [APPLE, SAMSUNG]

4. Find Even Numbers

Problem Statement: Given a list of integers, find all even numbers.

Java Program:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample4 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 15, 20, 25, 30);

        List<Integer> result = numbers.stream()
            .filter(n -> n % 2 == 0)
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

Expected Output: [10, 20, 30]

5. Find Numbers Greater Than 50

Problem Statement: Given a list of numbers, return only numbers greater than 50.

Java Program:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample5 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 60, 45, 80, 25, 100);

        List<Integer> result = numbers.stream()
```

```
        .filter(n -> n > 50)
        .collect(Collectors.toList());

    System.out.println(result);
}
}
```

Expected Output: [60, 80, 100]

6. Square Every Number

Problem Statement: Given a list of integers, create a new list containing the square of every number.

Java Program:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample6 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(2, 3, 4, 5);

        List<Integer> result = numbers.stream()
            .map(n -> n * n)
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

Expected Output: [4, 9, 16, 25]

7. Count Matching Elements

Problem Statement: Given a list of numbers, count how many numbers are greater than 50.

Java Program:

```
import java.util.Arrays;
import java.util.List;

public class StreamExample7 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 60, 45, 80, 25, 100);

        long count = numbers.stream()
            .filter(n -> n > 50)
            .count();

        System.out.println(count);
    }
}
```

Expected Output: 3

8. Find Maximum Number

Problem Statement: Given a list of integers, find the maximum value using Stream API.

Java Program:

```
import java.util.Arrays;
import java.util.List;

public class StreamExample8 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 60, 45, 80, 25);
```

```

        int max = numbers.stream()
            .max(Integer::compareTo)
            .get();

        System.out.println(max);
    }
}

```

Expected Output: 80

9. Find Minimum Number

Problem Statement: Given a list of integers, find the minimum value using Stream API.

Java Program:

```

import java.util.Arrays;
import java.util.List;

public class StreamExample9 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 60, 45, 80, 25);

        int min = numbers.stream()
            .min(Integer::compareTo)
            .get();

        System.out.println(min);
    }
}

```

Expected Output: 10

10. Sort Numbers

Problem Statement: Given a list of integers, sort the numbers in ascending order using Stream API.

Java Program:

```

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample10 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(50, 10, 40, 20, 30);

        List<Integer> result = numbers.stream()
            .sorted()
            .collect(Collectors.toList());

        System.out.println(result);
    }
}

```

Expected Output: [10, 20, 30, 40, 50]

11. Sort Strings

Problem Statement: Given a list of names, sort them alphabetically.

Java Program:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample11 {
    public static void main(String[] args) {

        List<String> names =
            Arrays.asList("Rahul", "Amit", "Swapnil");

        List<String> result = names.stream()
            .sorted()
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

Expected Output: [Amit, Rahul, Swapnil]

12. Remove Duplicate Values

Problem Statement: Given a list containing duplicate numbers, remove the duplicates.

Java Program:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample12 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 20, 10, 30, 20, 40);

        List<Integer> result = numbers.stream()
            .distinct()
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

Expected Output: [10, 20, 30, 40]

13. Find First Matching Element

Problem Statement: Given a list of names, find the first name whose length is greater than 5.

Java Program:

```
import java.util.Arrays;
import java.util.List;

public class StreamExample13 {
    public static void main(String[] args) {

        List<String> names =
```

```

        Arrays.asList("Amit", "Rahul", "Swapnil", "John");

String result = names.stream()
    .filter(s -> s.length() > 5)
    .findFirst()
    .orElse("Not Found");

System.out.println(result);
    }
}

```

Expected Output: Swapnil

14. Check Any Match

Problem Statement: Given a list of numbers, check whether any number is greater than 100.

Java Program:

```

import java.util.Arrays;
import java.util.List;

public class StreamExample14 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 20, 150, 40);

        boolean result = numbers.stream()
            .anyMatch(n -> n > 100);

        System.out.println(result);
    }
}

```

Expected Output: true

15. Check All Match

Problem Statement: Given a list of numbers, check whether all numbers are positive.

Java Program:

```

import java.util.Arrays;
import java.util.List;

public class StreamExample15 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 20, 30, 40);

        boolean result = numbers.stream()
            .allMatch(n -> n > 0);

        System.out.println(result);
    }
}

```

Expected Output: true

16. Sum of Numbers

Problem Statement: Given a list of integers, calculate the sum of all numbers.

Java Program:

```
import java.util.Arrays;
import java.util.List;

public class StreamExample16 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 20, 30, 40);

        int sum = numbers.stream()
            .mapToInt(Integer::intValue)
            .sum();

        System.out.println(sum);
    }
}
```

Expected Output: 100

17. Average of Numbers

Problem Statement: Given a list of integers, calculate the average value.

Java Program:

```
import java.util.Arrays;
import java.util.List;

public class StreamExample17 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 20, 30, 40);

        double average = numbers.stream()
            .mapToInt(Integer::intValue)
            .average()
            .orElse(0);

        System.out.println(average);
    }
}
```

Expected Output: 25.0

18. Convert List to Map

Problem Statement: Given a list of employee names, create a map where the key is the name and the value is the length of the name.

Java Program:

```
import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class StreamExample18 {
    public static void main(String[] args) {
```

```

        List<String> names =
            Arrays.asList("Amit", "Rahul", "Swapnil");

        Map<String, Integer> result = names.stream()
            .collect(Collectors.toMap(
                name -> name,
                String::length
            ));

        System.out.println(result);
    }
}

```

Expected Output: {Amit=4, Rahul=5, Swapnil=7}

19. Join Strings

Problem Statement: Given a list of names, join all names into one comma-separated string.

Java Program:

```

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamExample19 {
    public static void main(String[] args) {

        List<String> names =
            Arrays.asList("Amit", "Rahul", "Swapnil");

        String result = names.stream()
            .collect(Collectors.joining(", "));

        System.out.println(result);
    }
}

```

Expected Output: Amit, Rahul, Swapnil

20. Second Highest Number

Problem Statement: Given a list of numbers, find the second-highest distinct number using Stream API.

Java Program:

```

import java.util.Arrays;
import java.util.List;

public class StreamExample20 {
    public static void main(String[] args) {

        List<Integer> numbers =
            Arrays.asList(10, 50, 30, 80, 60, 80);

        int secondHighest = numbers.stream()
            .distinct()
            .sorted((a, b) -> b - a)
            .skip(1)
            .findFirst()
            .orElseThrow();

        System.out.println(secondHighest);
    }
}

```

Expected Output: 60

Stream API Quick Interview Cheat Sheet

<code>stream()</code>	→ Creates a stream from a collection
<code>filter()</code>	→ Selects elements matching a condition
<code>map()</code>	→ Transforms each element
<code>sorted()</code>	→ Sorts stream elements
<code>distinct()</code>	→ Removes duplicates
<code>count()</code>	→ Counts elements
<code>findFirst()</code>	→ Returns the first matching element
<code>anyMatch()</code>	→ Checks whether at least one element matches
<code>allMatch()</code>	→ Checks whether every element matches
<code>noneMatch()</code>	→ Checks whether no element matches
<code>min()/max()</code>	→ Finds minimum/maximum
<code>collect()</code>	→ Converts stream result into a collection
<code>joining()</code>	→ Joins strings
<code>mapToInt()</code>	→ Converts to <code>IntStream</code> for numeric operations
<code>sum()/average()</code>	→ Numeric aggregation
<code>skip()</code>	→ Skips first N elements
<code>limit()</code>	→ Keeps first N elements

Common interview pattern:

```
List<String> result = list.stream()
    .filter(condition)
    .map(transformation)
    .collect(Collectors.toList());
```

Interview tip: Be ready to explain the difference between `filter()` and `map()`, `stream()` vs `collection`, `intermediate` vs `terminal` operations, and why streams do not normally modify the original collection.